System architecture

A source-grounded map of the current implementation. One shared schedule; UTC storage; Denver business time.

React browser app

React 19 + TypeScript + React Router
/ HomePage
/services ServicesPage
/contact ContactPage + confirmation
Development: localhost:5173

Phoenix JSON API

Bandit HTTP server + Phoenix Endpoint
CORS, request parsing, router
Thin controllers + JSON renderers
GET services / availability
POST appointments
Development: localhost:4000

Domain + persistence

Catalog: services, sizes, pricing
Appointments: validated creation
Availability: time slots + overlaps
Repo: Ecto / Postgrex
PostgreSQL: schema + final constraints

Frontend responsibilities

Keep service/size/date/time and customer form state. Fetch catalog and availability. Convert Denver wall time to UTC for submission. Display confirmation or refreshed slot-conflict messaging.

Backend authority

Load price/duration from ServicePrice. Validate booking rules and serialize writers. Persist catalog price/duration snapshots. Return pending appointments; reject conflicting starts.

Runtime support

Application supervision: Telemetry, Repo, DNSCluster, Phoenix.PubSub and Endpoint under a one_for_one supervisor. Current inspected flow has no Google Calendar or payment integration. Confirmation means the pending request was saved, not a separate confirmation-status transition.

Evidence: frontend/src/{main.tsx,App.tsx}; backend/lib/scheduler_api_web/{endpoint.ex,router.ex}; backend/lib/scheduler_api/{application.ex,catalog.ex,appointments.ex}

Booking and recovery flow

The browser displays availability; appointment creation independently decides whether a slot can still be booked.

BROWSER	API	CONTEXT / DATABASE
1. Select service + size	GET /api/services	Catalog preloads prices + sizes
2. Choose a date	GET /api/availability	Generate valid local HH:MM slots
3. Pick time; fill form	POST /api/appointments	Validate input; lock; recheck; insert

Success: HTTP 201

Response: **{appointment: {...}}**
ContactPage stores createdAppointment and renders BookingConfirmation on the same route.

Displays customer name, service, size, vehicle, date, time, price, duration, status and confirmation ID. The saved status defaults to **pending**.

Race conflict: HTTP 422

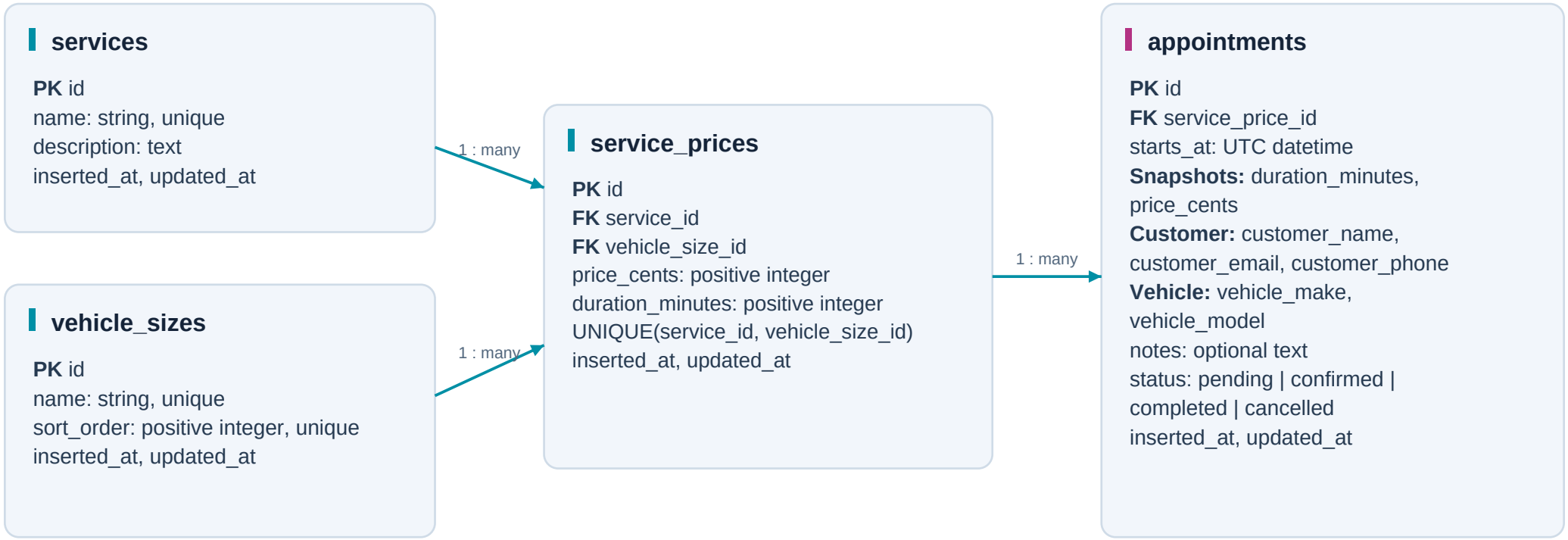
Response has a **starts_at** error. The UI clears selectedTime, refetches availability and removes unavailable options. Customer and vehicle values stay in state.

We apologize, but that slot was just claimed.
Another customer booked this time while you were completing your appointment. We've refreshed the available times below.

Evidence: frontend/src/App.tsx; frontend/src/components/BookingConfirmation.tsx; backend/lib/scheduler_api_web/controllers/{appointment_controller.ex,appointment_json.ex}; backend/lib/scheduler_api/appointments.ex

Database relationship map

Four domain tables. Pricing is attached to a service/vehicle-size pair; bookings reference that exact price row.



Data integrity

Foreign keys use restricted deletion. Required appointment fields are NOT NULL; customer/vehicle strings must be nonblank. Appointment indexes cover service_price_id, starts_at, and (status, starts_at). A partial GiST exclusion constraint prevents overlapping pending/confirmed occupied ranges, including the buffer.

Evidence: backend/lib/scheduler_api/catalog/{service.ex,vehicle_size.ex,service_price.ex}; backend/lib/scheduler_api/appointments/appointment.ex; backend/priv/repo/migrations/*.exs

Availability and concurrency

The same scheduling rules power both the informational GET endpoint and the authoritative pre-insert recheck.

Slot generation

Timezone: America/Denver via PostgreSQL IANA rules.
Days/hours: Monday-Saturday, 09:00-18:00.
Window: today through today + 90 days, inclusive.
Grid: 30-minute starts; same-day starts strictly future.

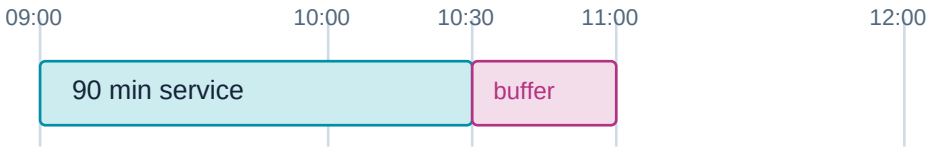
Load catalog duration. Build candidate starts. Keep only candidates whose duration + 30-minute buffer fits before closing. Exclude overlaps with pending/confirmed bookings.

One transaction, one shared lock

1. Validate customer-supplied input.
2. Begin transaction; acquire advisory lock.
3. Lock selected ServicePrice row FOR SHARE.
4. Copy catalog price/duration; recheck availability.
5. Insert; commit, or return a changeset error.

Advisory key: (0x4D41474E, 1). Waiting creators recheck after the first commit/rollback. Future multiple resources require coordinated key/query/constraint changes.

Occupied periods include the buffer



10:30 start conflicts.
11:00 start is allowed.

Overlap: requested_start < existing_end AND existing_start < requested_end.
Each end = starts_at + stored/trusted duration + 30 minutes. Ranges are [start, end), so touching endpoints are allowed. Cancelled/completed rows do not block. The exclusion constraint remains the final defense for writers that bypass the context.

Evidence: backend/lib/scheduler_api/appointments/{availability.ex,appointment.ex}; backend/lib/scheduler_api/appointments.ex; backend/priv/repo/migrations/20261019222804_prevent_overlapping_appointments.exs

API contract and ownership

All public API routes use the JSON pipeline. No appointment listing, update or delete HTTP route is currently defined.

GET /api/services

200: array of services with prices.

Service: id, name, description.
Price option: service_price_id, price_cents, duration_minutes, vehicle_size, vehicle_size_id.

Owner: Catalog context.

GET /api/availability

Query: date=YYYY-MM-DD and service_price_id.
200: date, service_price_id, duration_minutes, available_times.
Times use Denver-local HH:MM.
No slots: available_times: [].
Owner: Availability module.

POST /api/appointments

Required: service_price_id, starts_at, customer_name, customer_email, customer_phone, vehicle_make, vehicle_model.
Optional: notes.
201: {appointment: {...}}
Owner: Appointments context.

Validation and errors

Availability request errors

422: invalid_date, past_date, outside_booking_window, closed_sunday, invalid_service_price_id.
404: not_found (unknown service price).

`{error: {field, code, message}}`
Malformed, missing and oversized IDs are rejected before the service-price query.

Appointment validation errors

422: {errors: {field: [messages]}}
Slot conflict: starts_at contains **is not an available start time** or **is no longer available** (DB constraint).

Client price, duration and status are ignored. The backend normalizes contact fields, validates input, and snapshots catalog values. The default saved status is pending.

Evidence: backend/lib/scheduler_api_web/router.ex; backend/lib/scheduler_api_web/controllers/{service_controller.ex,service_json.ex,availability_controller.ex,availability_json.ex,appointment_controller.ex,appointment_json.ex}

Environments, tests and runbook

Database names identify separate PostgreSQL databases, not separate tables. Each contains the same migrated domain schema.

Development

React: localhost:5173
Phoenix: localhost:4000
Database: scheduler_api_dev
Real development bookings persist.
Frontend API origin is fixed to port 4000.

Backend tests

MIX_ENV: test
Database: scheduler_api_test
ExUnit + Ecto SQL Sandbox.
Context, availability and API tests.
Concurrency tests use independent SQL connections and explicit cleanup.

Playwright E2E

React: 127.0.0.1:5174
Phoenix: 127.0.0.1:4002
Database: scheduler_api_test_e2e
MIX_ENV=test; partition=_e2e.
One worker; real API responses.

E2E isolation and coverage

Harness forwards browser API calls to the test Phoenix server. Fixtures create E2E Mid Detail / E2E Small, choose the next open Denver date, then clean up appointments and catalog rows in finally. Covers booking confirmation and a competing real API booking; stale setup fixtures are removed.

Code ownership guide

UI + form/requests: frontend/src/App.tsx
Success screen: src/components/BookingConfirmation.tsx
Business logic: backend/lib/scheduler_api/
HTTP + rendering: backend/lib/scheduler_api_web/
Schema history: backend/priv/repo/migrations/

Commands (run inside the indicated directory)

backend/

```
mix setup
mix phx.server
mix test
mix compile --warnings-as-errors
mix precommit
```

frontend/

```
npm ci
npm run dev
npm run lint
npm run build
npm run test:e2e
```

One-time browser install

```
npx playwright install chromium
```

Runtimes (mise.toml)

Elixir 1.20.2 / Erlang 28.4 / Node 24
E2E auto-creates/migrates its test DB.

Evidence: backend/config/{dev.exs,test.exs}; frontend/{package.json,playwright.config.ts,e2e/fixtures.ts,e2e/booking.spec.ts}; backend/scripts/e2e.exs; mise.toml